

USER GUIDE for Implementing and Using the Olympic Games Database

Assignment: Database Systems – ISYS2014/ISYS5008 2024 Semester 2

Assessment: User Guide to Implement and Use the Database

Student Name: Goh Soon Hing

Curtin Student ID: 22488400 (700048913)

Lab Group: Group 2

INTRODUCTION

This guide will show you how to set up and utilize the Olympic Games database in a MySQL server. By following the instructions, you will be able to construct the database, run the SQL queries created for the project, and run the scripts to create tables. Before proceed, ensure that MySQL has been properly installed and configured on your system.

- **Database Name:** OlympicGames_22488400
- **MYSQL Version:** MySQL 8.0.39
- **Operating System:** Ubuntu 22.04.1

PRE-REQUIREMENT

Before proceeding, please ensure the following:

- MySQL is already installed on your machine.
- The command line for MySQL is available to you.
- You possess the rights required to create MySQL tables and databases.

DATABASE CREATION

Step 1: Log in to MYSQL by following command:

```
Mysql -u root -p
```

Step 2: Create the Database

```
CREATE DATABASE OlympicGames_22488400;  
USE OlympicGames_22488400;
```

Step 3: Validate Database Creation

```
SHOW DATABASES;
```

Sample:

```
--create sample database  
CREATE DATABASE OlympicGames_22488400;  
  
--using database  
USE OlympicGames_22488400;
```

CREATING TABLES:

Sample:

```
--create Game Table
CREATE TABLE Games
(
    GameID INT PRIMARY KEY NOT NULL,
    Year INT NOT NULL,
    Season VARCHAR(10) NOT NULL,
    City VARCHAR(50) NOT NULL
);

--create Event table
CREATE TABLE Event
(
    EventID INT PRIMARY KEY NOT NULL,
    EventName VARCHAR(100) NOT NULL,
    Sport VARCHAR(50) NOT NULL,
    Discipline VARCHAR(50),
    Gender VARCHAR(10) NOT NULL,
    GameID INT NOT NULL,
    FOREIGN KEY (GameID) REFERENCES Games(GameID)
);

--create Country table
CREATE TABLE Country
(
    CountryCode CHAR(3) PRIMARY KEY NOT NULL,
    CountryName VARCHAR(50) NOT NULL
);

--create Participant table
CREATE TABLE Participant
(
    ParticipantID INT PRIMARY KEY NOT NULL,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Gender CHAR(1) NOT NULL,
    DateOfBirth DATE NOT NULL,
    CountryCode CHAR(3) NOT NULL,
    FOREIGN KEY (CountryCode) REFERENCES Country(CountryCode)
);
```

```
--create Team table
CREATE TABLE Team
(
    TeamID INT PRIMARY KEY NOT NULL,
    TeamName VARCHAR(100) NOT NULL,
    CountryCode CHAR(3) NOT NULL,
    FOREIGN KEY (CountryCode) REFERENCES Country(CountryCode)
);

--create Participation table
CREATE TABLE Participation
(
    ParticipationID INT PRIMARY KEY NOT NULL,
    EventID INT NOT NULL,
    ParticipantID INT,
    TeamID INT,
    Position INT,
    Result VARCHAR(50),
    FOREIGN KEY (EventID) REFERENCES Event(EventID),
    FOREIGN KEY (ParticipantID) REFERENCES Participant(ParticipantID),
    FOREIGN KEY (TeamID) REFERENCES Team(TeamID)
);

--create Medal Table
CREATE TABLE Medal
(
    MedalID INT PRIMARY KEY NOT NULL,
    MedalType VARCHAR(10) NOT NULL,
    EventID INT NOT NULL,
    ParticipantID INT,
    TeamID INT,
    FOREIGN KEY (EventID) REFERENCES Event(EventID),
    FOREIGN KEY (ParticipantID) REFERENCES Participant(ParticipantID),
    FOREIGN KEY (TeamID) REFERENCES Team(TeamID)
);

--show tables
SHOW TABLES;
```

INSERT DATA INTO TABLES:

Sample:

```
--insert data into Game table
INSERT INTO Games (GameID, Year, Season, City) VALUES
(1, 2020, 'Summer', 'Tokyo'),
(2, 2016, 'Summer', 'Rio de Janeiro'),
(3, 2012, 'Summer', 'London'),
(4, 2008, 'Summer', 'Beijing'),
(5, 2004, 'Summer', 'Athens'),
(6, 2000, 'Summer', 'Sydney'),
(7, 1996, 'Summer', 'Atlanta'),
(8, 1992, 'Summer', 'Barcelona'),
(9, 1988, 'Summer', 'Seoul'),
(10, 1984, 'Summer', 'Los Angeles'),
(11, 1980, 'Summer', 'Moscow'),
(12, 1976, 'Summer', 'Montreal'),
(13, 1972, 'Summer', 'Munich'),
(14, 1968, 'Summer', 'Mexico City'),
(15, 1964, 'Summer', 'Tokyo'),
(16, 1960, 'Summer', 'Rome'),
(17, 1956, 'Summer', 'Melbourne'),
(18, 1952, 'Summer', 'Helsinki'),
(19, 1948, 'Summer', 'London'),
(20, 1936, 'Summer', 'Berlin');

--verify inserted data
SELECT * FROM Games;

--insert sample data into Event table
INSERT INTO Event (EventID, EventName, Sport, Discipline, Gender, GameID) VAL
(1, '100m Sprint', 'Athletics', 'Track', 'Men', 1),
(2, '200m Sprint', 'Athletics', 'Track', 'Women', 1),
(3, 'Long Jump', 'Athletics', 'Field', 'Men', 1),
(4, 'High Jump', 'Athletics', 'Field', 'Women', 1),
(5, '400m Relay', 'Athletics', 'Track', 'Men', 2),
(6, 'Marathon', 'Athletics', 'Long Distance', 'Women', 2),
(7, 'Javelin Throw', 'Athletics', 'Field', 'Men', 3),
(8, '800m', 'Athletics', 'Track', 'Women', 3),
(9, '100m Butterfly', 'Swimming', 'Butterfly', 'Men', 3),
(10, '200m Freestyle', 'Swimming', 'Freestyle', 'Women', 4),
(11, 'Diving', 'Aquatics', 'Springboard', 'Men', 4),
(12, 'Shooting', 'Shooting', 'Air Rifle', 'Women', 4),
(13, 'Cycling', 'Cycling', 'Road Race', 'Men', 5),
(14, 'Gymnastics', 'Gymnastics', 'Artistic', 'Women', 5),
```



```
(15, 'Tennis Singles', 'Tennis', 'Hard Court', 'Men', 5),
(16, 'Weightlifting', 'Weightlifting', 'Heavyweight', 'Men', 6),
(17, 'Boxing', 'Boxing', 'Lightweight', 'Men', 6),
(18, 'Archery', 'Archery', 'Recurve', 'Women', 7),
(19, 'Football Final', 'Football', 'Outdoor', 'Men', 7),
(20, 'Basketball Final', 'Basketball', 'Indoor', 'Women', 8);
```

--verify inserted data

```
SELECT * FROM Event;
```

--insert sample data into country table

```
INSERT INTO Country (CountryCode, CountryName) VALUES
```

```
('JAM', 'Jamaica'),
('USA', 'United States of America'),
('GBR', 'Great Britain'),
('RUS', 'Russia'),
('AUS', 'Australia'),
('KEN', 'Kenya'),
('BRA', 'Brazil'),
('ARG', 'Argentina'),
('SRB', 'Serbia'),
('ESP', 'Spain'),
('RSA', 'South Africa'),
('CHN', 'China'),
('JPN', 'Japan'),
('GER', 'Germany'),
('CAN', 'Canada'),
('FRA', 'France'),
('ITA', 'Italy'),
('FIN', 'Finland'),
('SWE', 'Sweden'),
('NOR', 'Norway');
```

--verify inserted data

```
SELECT * FROM Country;
```

--insert sample data into Participant table

```
INSERT INTO Participant (ParticipantID, FirstName, LastName, Gender, DateOfBirth)
```

```
(1, 'Michael', 'Smith', 'M', '1988-01-15', 'USA'),
(2, 'Jessica', 'Jones', 'F', '1990-02-22', 'GBR'),
(3, 'Robert', 'Johnson', 'M', '1989-03-12', 'BRA'),
(4, 'Sophia', 'Williams', 'F', '1994-04-05', 'ARG'),
(5, 'Liam', 'Brown', 'M', '1991-05-18', 'KEN');
```

```
--insert sample data into Team table
INSERT INTO Team (TeamID, TeamName, CountryCode) VALUES
(1, 'Jamaica Sprint Team', 'JAM'),
(2, 'USA Track Team', 'USA'),
(3, 'Great Britain Marathon Team', 'GBR'),
(4, 'Russia Pole Vault Team', 'RUS'),
(5, 'Australia Relay Team', 'AUS'),
(6, 'Kenya Marathon Team', 'KEN'),
(7, 'Brazil Football Team', 'BRA'),
(8, 'Argentina Football Team', 'ARG'),
(9, 'Serbia Tennis Team', 'SRB'),
(10, 'USA Gymnastics Team', 'USA'),
(11, 'USA Swimming Team', 'USA'),
(12, 'Jamaica Relay Team', 'JAM'),
(13, 'Finland Distance Team', 'FIN'),
(14, 'South Africa Rugby Team', 'RSA'),
(15, 'Germany Cycling Team', 'GER'),
(16, 'France Basketball Team', 'FRA'),
(17, 'Italy Volleyball Team', 'ITA'),
(18, 'Japan Judo Team', 'JPN'),
(19, 'Spain Basketball Team', 'ESP'),
(20, 'Norway Ski Team', 'NOR');

--verify inserted data
SELECT * FROM Team;
```

ADVANCED DATABASE OPERATIONS (Stored Procedure, Views and Indexes)

Sample:

```
#1: Stored Procedures

--1a: Stored Procedure to Update Participant's Team Participation
--This stored procedure updates the team participation for a participant in a specific event.
DELIMITER //

CREATE PROCEDURE UpdateTeamParticipation (
    IN pParticipantID INT,
    IN pEventID INT,
    IN pNewTeamID INT
)
BEGIN
    -- Check if the Participant exists in the given event
    IF EXISTS (SELECT 1 FROM Participation WHERE ParticipantID = pParticipantID AND EventID = pEventID) THEN
        -- Update the TeamID in the Participation table
        UPDATE Participation
        SET TeamID = pNewTeamID
        WHERE ParticipantID = pParticipantID AND EventID = pEventID;
    ELSE
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'No participation record found for this Participant and Event.';
    END IF;
END //

DELIMITER ;
--Usage:
CALL UpdateTeamParticipation(1, 1, 12); -- Update Usain Bolt's participation in event 1 to be part of Team 12

--1b: Stored Procedure to Get Medal Count by Country
--This stored procedure retrieves the total number of medals won by each country
DELIMITER //
CREATE PROCEDURE GetMedalCountByCountry (IN pCountryCode CHAR(3))
BEGIN
    SELECT C.CountryName, COUNT(M.MedalID) AS MedalCount
    FROM Medal M
    JOIN Participant P ON M.ParticipantID = P.ParticipantID
    JOIN Country C ON P.CountryCode = C.CountryCode
    WHERE C.CountryCode = pCountryCode
    GROUP BY C.CountryName;
END //
DELIMITER ;
```

```

--Usage:
CALL GetMedalCountByCountry('USA');

#2: Views

--2a: Medal Summary by Country
--Purpose: This view is useful to easily retrieve how many gold, silver, and bronze medals each country has won with
CREATE VIEW MedalSummaryByCountry AS
SELECT
    c.CountryCode,
    c.CountryName,
    SUM(CASE WHEN m.MedalType = 'Gold' THEN 1 ELSE 0 END) AS TotalGoldMedals,
    SUM(CASE WHEN m.MedalType = 'Silver' THEN 1 ELSE 0 END) AS TotalSilverMedals,
    SUM(CASE WHEN m.MedalType = 'Bronze' THEN 1 ELSE 0 END) AS TotalBronzeMedals,
    COUNT(m.MedalID) AS TotalMedals
FROM
    Country c
LEFT JOIN
    Participant p ON c.CountryCode = p.CountryCode
LEFT JOIN
    Medal m ON p.ParticipantID = m.ParticipantID
GROUP BY
    c.CountryCode, c.CountryName;

--2b: Participation with No Medals
--Purpose: This view allows you to quickly find participants who have not won any medals. It can be useful for analyzing
CREATE VIEW ParticipantsWithoutMedals AS
SELECT
    p.ParticipantID,
    p.FirstName,
    p.LastName,
    p.CountryCode,
    c.CountryName
FROM
    Participant p
LEFT JOIN
    Medal m ON p.ParticipantID = m.ParticipantID
JOIN
    Country c ON p.CountryCode = c.CountryCode
WHERE
    m.MedalID IS NULL;

#3: Indexes

--3a: Index on CountryCode in the Participant Table
--Purpose: Speed up queries filtering by country.
CREATE INDEX idx_participant_country ON Participant (CountryCode);

--3b: Index on EventID in the Participation Table
--Purpose: Enhance performance for queries accessing participation by event.
CREATE INDEX idx_participation_event ON Participation (EventID);

--Usage:
-- Retrieve all participants from Jamaica
SELECT * FROM Participant WHERE CountryCode = 'CHN';

-- Retrieve all participation records for a specific event
SELECT * FROM Participation WHERE EventID = 1;

```


GUIDE FOR USING DATABASE

Firstly you need to compile the python script (mysql_connect) by `python3 mysql_connect.py` command. After that you would see a user input menu consists of view all tables, view record, insert, update, delete data and exit from system.

Sample:

```
Main Menu:
1. List all tables
2. View records from a table
3. Insert a new record
4. Update an existing record
5. Delete a record
0. Exit
Choose an option: █
```

Then, the system would ask user enter the table option that want be viewed or others function.

```
Choose an option: 2
Enter the table name to view records: █
```

CONCLUSION

By following the instruction, user should be able to successfully construct the OlympicGames_22488400 database, create and populate tables, execute the queries written. The scripts offered should make it easier to implement and use databases.